SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1G56)
ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE s ENGINEERING

Name : THULASIPRASATH.T

Reg.No : 11249A383

Class : II B.E(CSE)

Subject Code : BCSF244A12

Subject Name : DATA SCIENCE TOOLKIT LAB

SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1G56)
ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE s ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by

Mr/Ms. THULASIPRASATH.T
with Reg. No 11249A383 done by of II-B.E.(CSE) in DATA SCIENCE TOOLKIT LAB (BCSF244A12) for Lab during the academic year 2025 -2026.

Station : ENATHUR

Date :

Staf-in-charge Head of the Department

Submitted for the Practical examination held on_____

Examine-1 Examine -2

INDEX

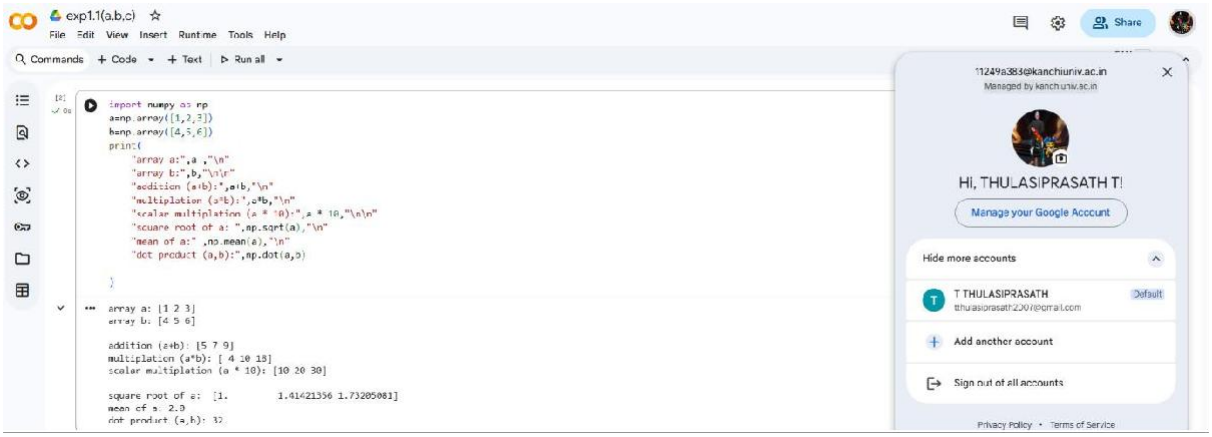
SI.NO	Program Name	Date	Page .no	Signature
01 (1.1)	Introduction to python libraries for Data Science.	02.02.2026	01	
1.2	pandas	02.02.2026	02	
1.3	Matplot lib	02.02.2026	04	
1.4	Scikit learn	06.02.2026	06	
02	Perform dataexploration and preprocessing in python.	14.02.2026	07	
03	Implementregularized linear regression.	14.02.2026	09	
04	Implement naïvebayes classifier.	23.02.2026	10	
05	Implementregularized logistic regression.	02.03.2026	11	
06	Build modelsusing different Ensembling Techniques.	06.03.2026	13	
07	Build models using Decision Trees.	06.03.2026	14	
08	Build the models using different Kernel Functions.	16.03.2026	15	
06	Implement K-Nearest Naighbors (KNN) classification.	30.03.2026	16	
10	K-Means clustering with PCA and Elbow Method.	30.03.2026	17	

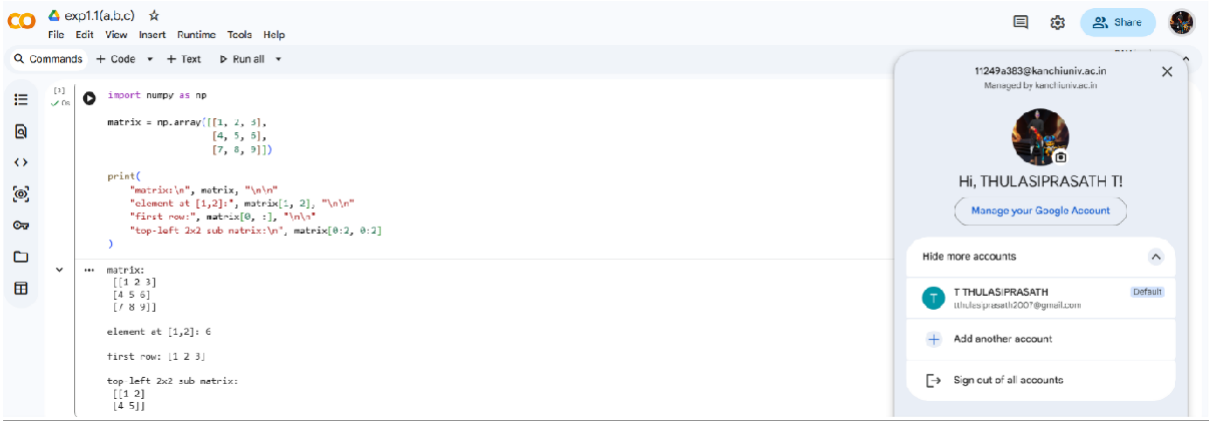
Exp.no: 01 (1.1)
Date : 02.02.2026

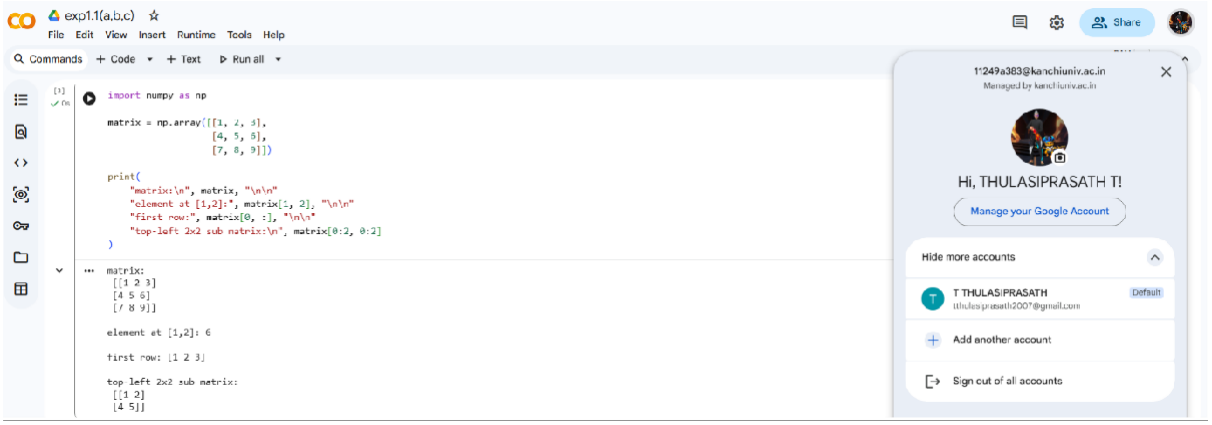
INTRODUCTION TO PYTHON LIBRARIES FOR DATASCIENCE.

AIM:

To study and use basic Python libraries like pandas, Matplotlib, and Scikit-learn for data analysis.







Exp.no: 1.2
Date : 02.02.2026

PANDAS.

exp 1.2(a,b,c)

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
import pandas as pd

# From a Dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Paris', 'London']
}

df = pd.DataFrame(data)

# Viewing data
print("HEAD:")
print(df.head())   # first 5 rows

print("\nINFO:")
print(df.info())   # Summary

print("\nDESCRIBE:")
print(df.describe())   # Statistical summary


# Selection and Filtering
ages = df['Age']
print("\nAGES COLUMN:")
print(ages)

above_25 = df[df['Age'] > 25]
print("\nAGE > 25:")
print(above_25)
```

Share

11249a383@kanchiuniv.ac.in
Managed by kanchiuniv.ac.in

Hi, THULASIPRASATH T!

Manage your Google Account

Hide more accounts

T THULASIPRASATH Default
thulasiprasath2007@gmail.com

Add another account

Sign out of all accounts

Privacy Policy • Terms of Service

exp 1.2(a,b,c)

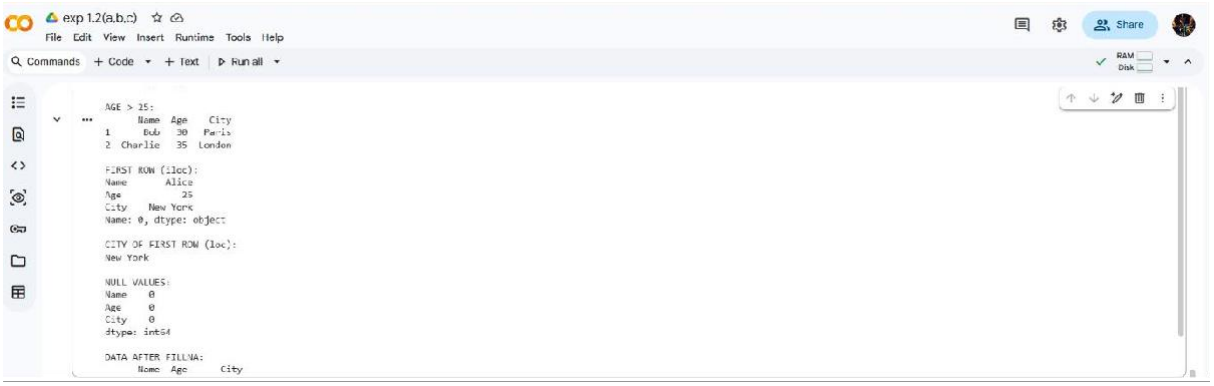
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

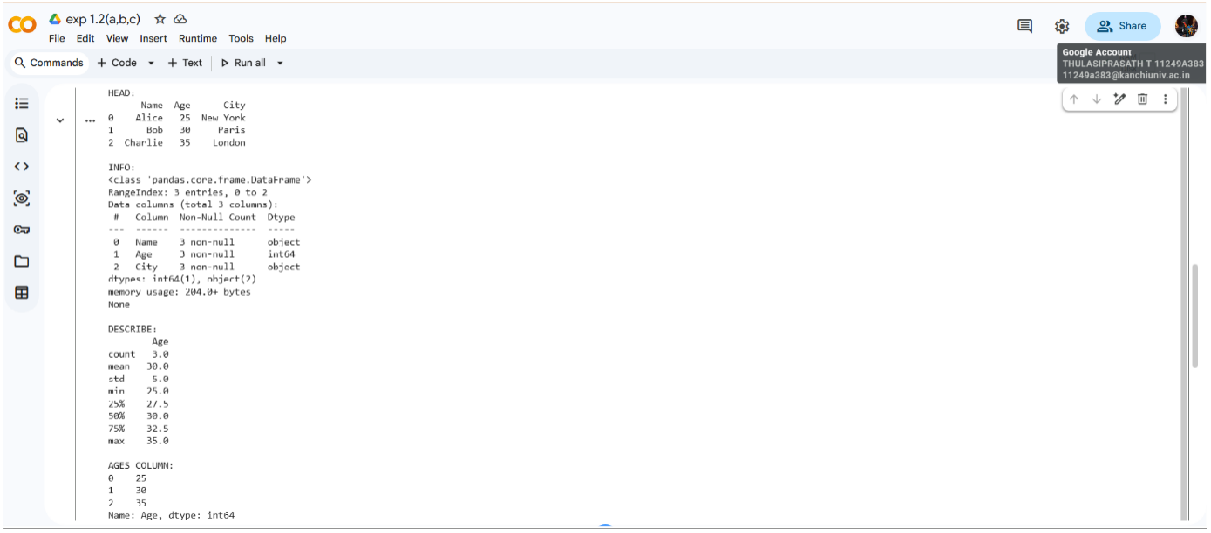
Share

```python
# iloc vs loc
row_0 = df.iloc[0]
print("\nFIRST ROW (iloc):")
print(row_0)

specific_val = df.loc[0, 'City']
print("\nCITY OF FIRST ROW (loc):")
print(specific_val)


# Data Cleaning
print("\nNULL VALUES:")
print(df.isnull().sum())

df_clean = df.dropna()

df_filled = df.fillna(0)
print("\nDATA AFTER FILLNA:")
print(df_filled)
```

OUTPUT:



```
AGE > 25:
      Name  Age    City
1      Bob   30   Paris
2  Charlie   35  London

FIRST ROW (iloc):
Name       Alice
Age           25
City    New York
Name: 0, dtype: object

CITY OF FIRST ROW (loc):
New York

NULL VALUES:
Name    0
Age     0
City    0
dtype: int64

DATA AFTER FILLNA:
     Name  Age      City
```



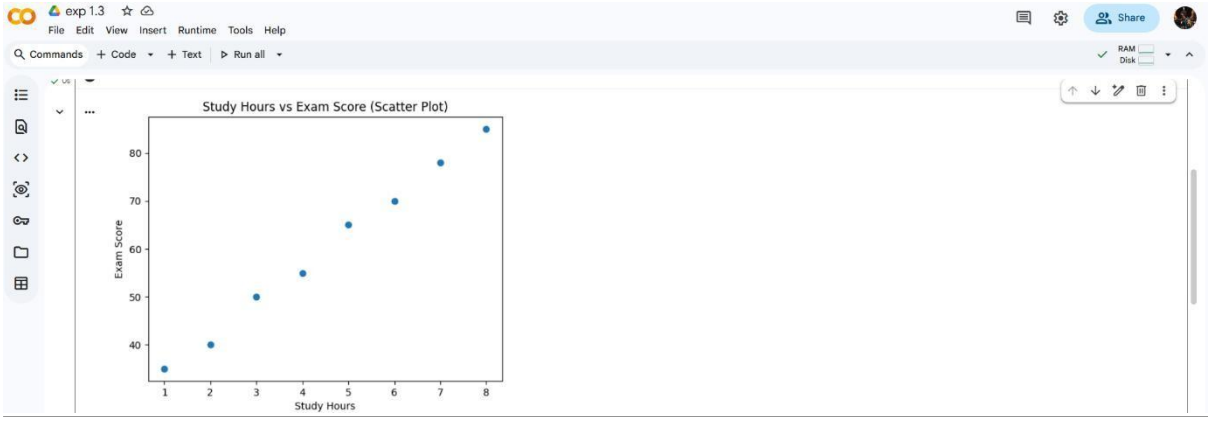
```
HEAD:
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London

INFO:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None

DESCRIBE:
             Age
count   3.0000
mean   30.0000
std     5.0000
min    25.0000
25%    27.5000
50%    30.0000
75%    32.5000
max    35.0000

AGES COLUMN:
0    25
1    30
2    35
Name: Age, dtype: int64
```

Exp.no: 1.3
Date : 02.02.2026

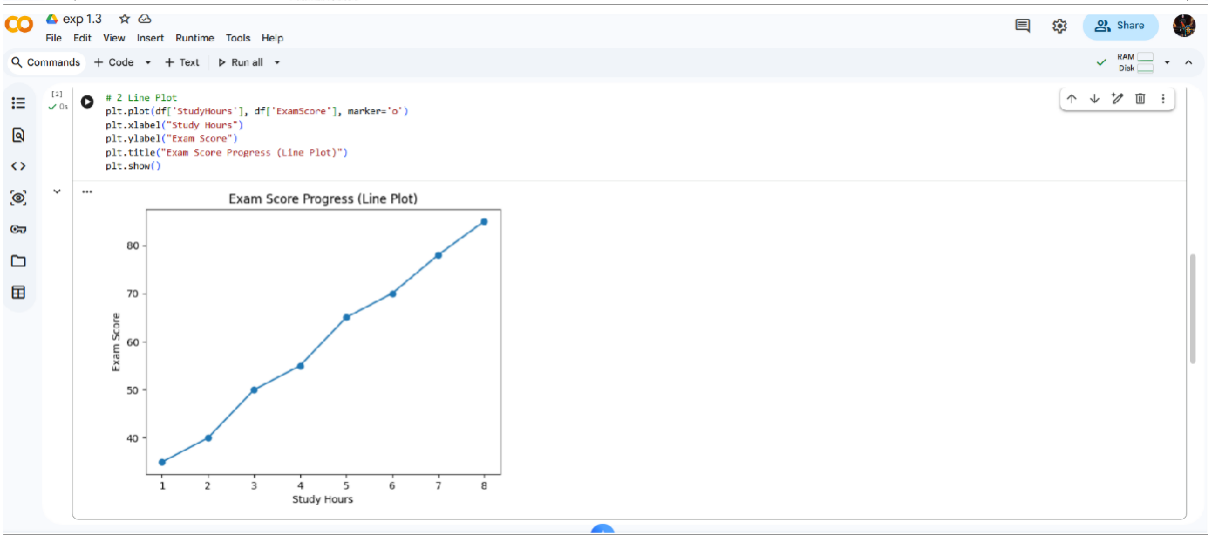
MATPLOT LIB.

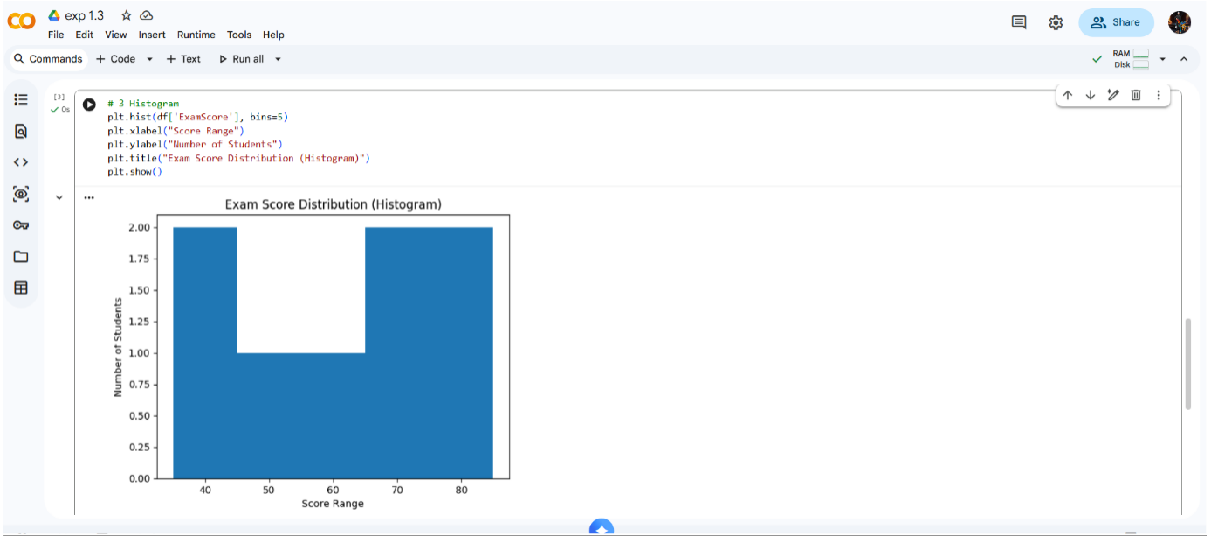
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Create dataset
data = {
    'StudyHours': [1, 2, 3, 4, 5, 6, 7, 8],
    'ExamScore': [35, 40, 50, 55, 65, 70, 78, 85]
}

df = pd.DataFrame(data)

# 1 scatter Plot
plt.scatter(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Scatter Plot)")
plt.show()
```

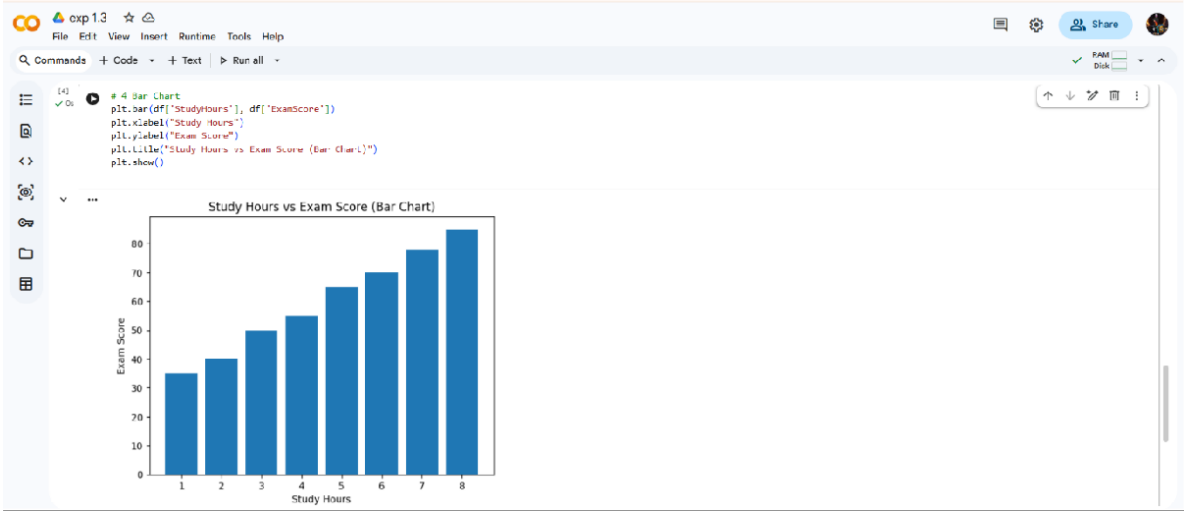
OUTPUT:




```
# 2 Line Plot
plt.plot(df['StudyHours'], df['ExamScore'], marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Exam Score Progress (Line Plot)")
plt.show()
```




exp 1.3
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
# 3 Histogram
plt.hist(df['ExamScore'], bins=5)
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.title("Exam Score Distribution (Histogram)")
plt.show()
Exam Score Distribution (Histogram)
Number of Students
Score Range


exp 1.3
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
# 4 Bar Chart
plt.bar(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Bar Chart)")
plt.show()
Study Hours vs Exam Score (Bar Chart)
Exam Score
Study Hours

Exp.no: 1.4
Date : 09.02.2026

SCIKIT LEARN.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
data = {
'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
```

```
  Student  Classes_Attended  Internal_Marks
0      S1                30              35
1      S2                35              38
2      S3                40              42
3      S4                45              46
4      S5                50              50
5      S6                55              55
6      S7                60              60
7      S8                65              65
8      S9                70              70
9     S10                75              75
```

```
X = df[['Classes_Attended']]
y = df['Internal_Marks']
```

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=7
)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

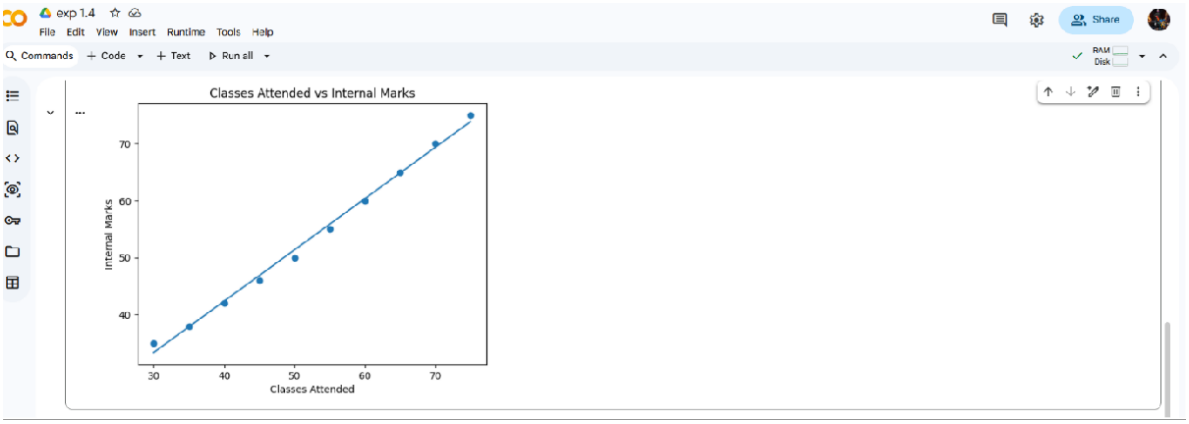
```
predictions = model.predict(X_test)
```

```
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)
```

```
MSE: 0.578125
Marks per Class: 0.5
Base Marks: 6.375
```

```
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

OUTPUT:



RESULT:

Successfully understood and implemented basic operations using essential Python data science libraries.

Exp.no: 02
Date : 14.02.2026

PERFORM DATA OPERSTION AND PREPROCESING IN PYTHON.

AIM:

To perform data cleaning, transformation, and exploratory data analysis on a dataset.

exp 2

11249A383 THULASIPRASATH T

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
# --- STEP 1: LOAD DATA(creating dummy dataset for the lab)---
data = {
    'country':['India','USA','India','USA ,'UK','India'],
    'Age':[22,25,np.nan,30,28,35],#note the 'nan' (missing value)
    'Salary':[40000,60000,50000,np.nan,72000,58000],
    'Purchased':['No','Yes ,'Yes','No','Yes','Yes']

    }

df =pd.DataFrame(data)
print("---ORIGINAL RAW DATA---")
print(df)
print("\n")

# --- STEP 2: INSPECT DATA ---

# Check for missing values
print("--- MISSING VALUES COUNT ---")
print(df.isnull().sum())
print("\n")
```

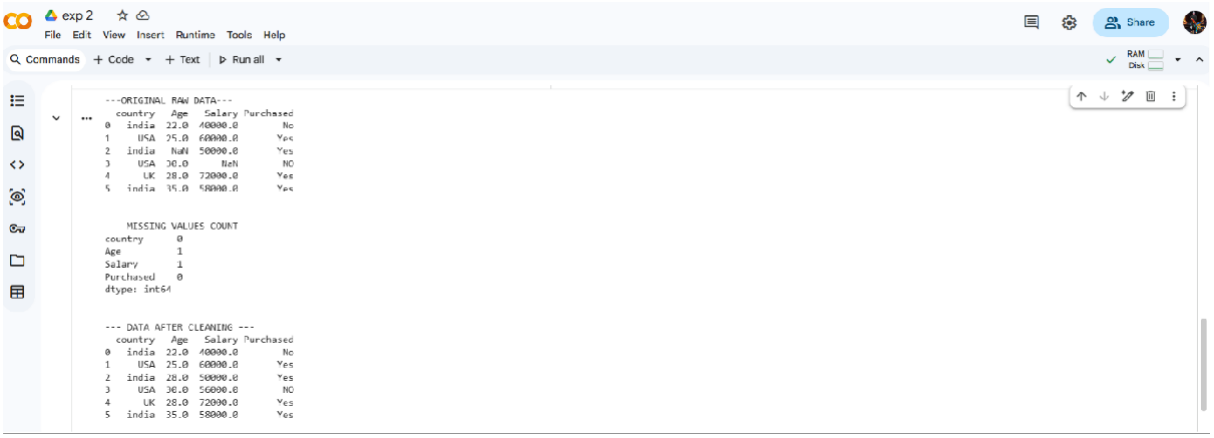
exp 2

```python
# --- STEP 3: CLEAN DATA (Handling Missing Values) ---

# Logic: Fill missing Age/Salary with the Average (Mean) of that column
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())

print("--- DATA AFTER CLEANING ---")
print(df)
print("\n")


# --- STEP 4: CONVERT TEXT TO NUMBERS (Encoding) ---

# Logic: Machines can't read 'India' or 'USA'. We convert them to numbers.
# We use 'One Hot Encoding' (get_dummies)

df_encoded = pd.get_dummies(df, columns=['country'])

# For the Target column (Purchased), let's map Yes/No manually
df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})

print("--- DATA AFTER ENCODING (All Numbers Now)---")
print(df_encoded)
print("\n")
```

exp 2

```python
print("\n")


# --- STEP 5: SCALE FEATURES ---

# Logic: Age is 20-30, Salary is 40000-60000. Salary dominates Age because it's bigger.
# So we scale features to bring them to the same range.

from sklearn.preprocessing import StandardScaler

# Separate Features (X) and Target (y)
X = df_encoded.drop('Purchased', axis=1)
y = df_encoded['Purchased']

# Initialize StandardScaler
scaler = StandardScaler()

# Fit and Transform
X_scaled = scaler.fit_transform(X)

print("--- DATA AFTER SCALING ---")
print(X_scaled)
print("\n")
```

OUTPUT:



```
---ORIGINAL RAW DATA---
  country   Age   Salary Purchased
0   india  22.0  40000.0        No
1     USA  25.0  60000.0       Yes
2   india   NaN  50000.0       Yes
3     USA  30.0      NaN        NO
4      UK  28.0  72000.0       Yes
5   india  35.0  58000.0       Yes


    MISSING VALUES COUNT
country      0
Age          1
Salary       1
Purchased    0
dtype: int64


--- DATA AFTER CLEANING ---
  country   Age   Salary Purchased
0   india  22.0  40000.0        No
1     USA  25.0  60000.0       Yes
2   india  28.0  50000.0       Yes
3     USA  30.0  56000.0        NO
4      UK  28.0  72000.0       Yes
5   india  35.0  58000.0       Yes
```

```
--- DATA AFTER ENCODING (All Numbers Now)---
    Age   Salary  Purchased  country_UK  country_USA  country_india
0  22.0  40000.0        0.0       False        False           True
1  25.0  60000.0        1.0       False         True          False
2  28.0  50000.0        1.0       False        False           True
3  30.0  56000.0        NaN       False         True          False
4  28.0  72000.0        1.0        True        False          False
5  35.0  58000.0        1.0       False        False           True


--- DATA AFTER SCALING ---
[[-1.48461498 -1.5444529  -0.4472136  -0.70710678  1.        ]
 [-0.74236749  0.41111323 -0.4472136   1.41421356 -1.        ]
 [ 0.         -0.81666984 -0.4472136  -0.70710678  1.        ]
 [ 0.40487165  0.          0.4472136   1.41421356  1.        ]
 [ 0.          1.5444529  2.23606798 -0.70710678 -1.        ]
 [ 1.73205081  0.20555661 -0.4472136  -0.70710678  1.        ]]
```

RESULT:

Data was preprocessed and explored effectively, making it suitable for model building.

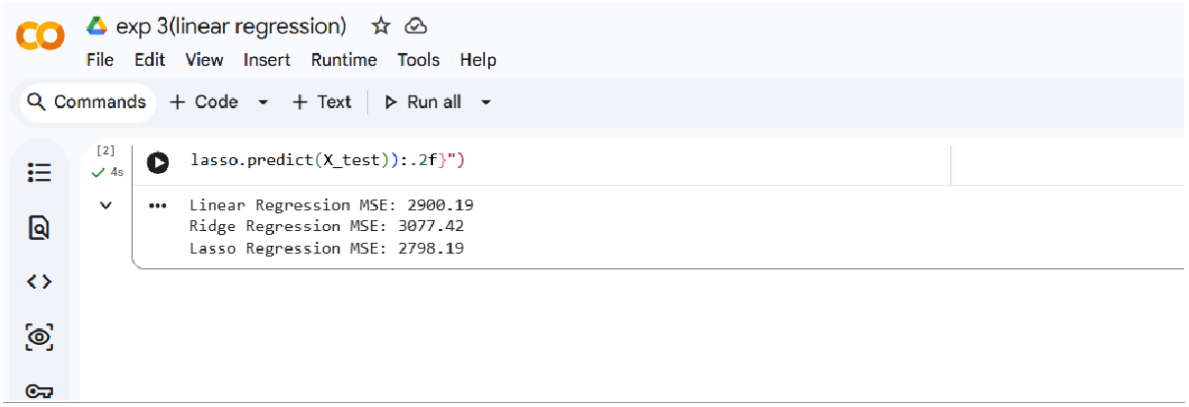
Exp.no: 03
Date : 14.02.2026

IMPLEMENTED REGULARIZED LINEAR REGRESSION.

AIM:

PROGRAM:

OUTPUT:



RESULT:

Successfully implemented the Built a regression model with improved performance and reduced overfitting.

Exp.no: 04
Date : 23.02.2026

IMPLEMENT NAIVE BAYES CLASSIFIER.

AIM:

To implement the Naïve Bayes algorithm for classification tasks.

PROGRAM:

OUTPUT:

RESULT:

Successfully Developed a classifier capable of predicting class labels with reasonable accuracy.

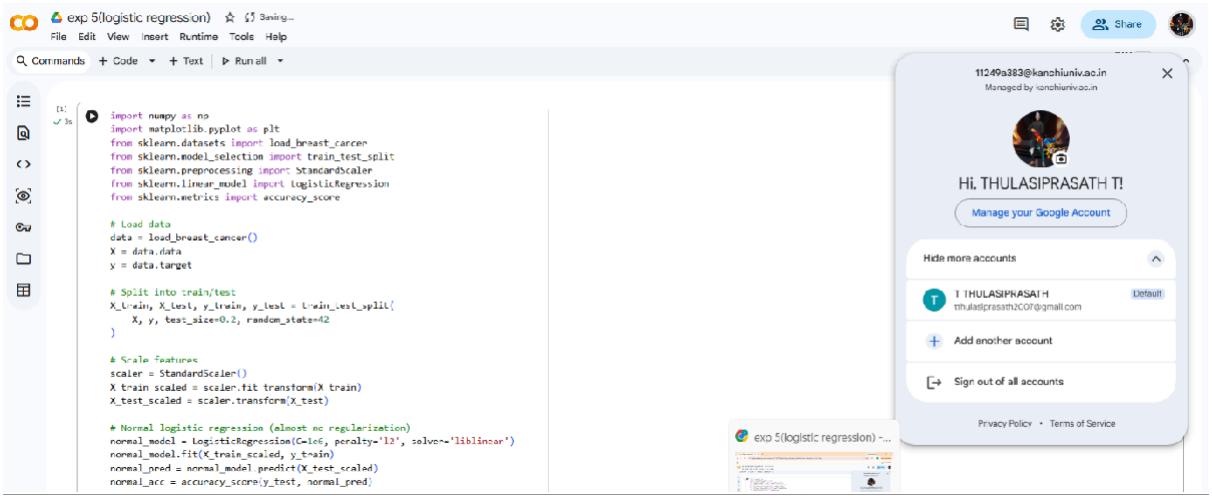
Exp.no: 05
Date : 02.03.2026

IMPLEMENT REGULARIZED LOGISTIC REGRESSION.

AIM:

To implement logistic regression with regularization for classification problems.

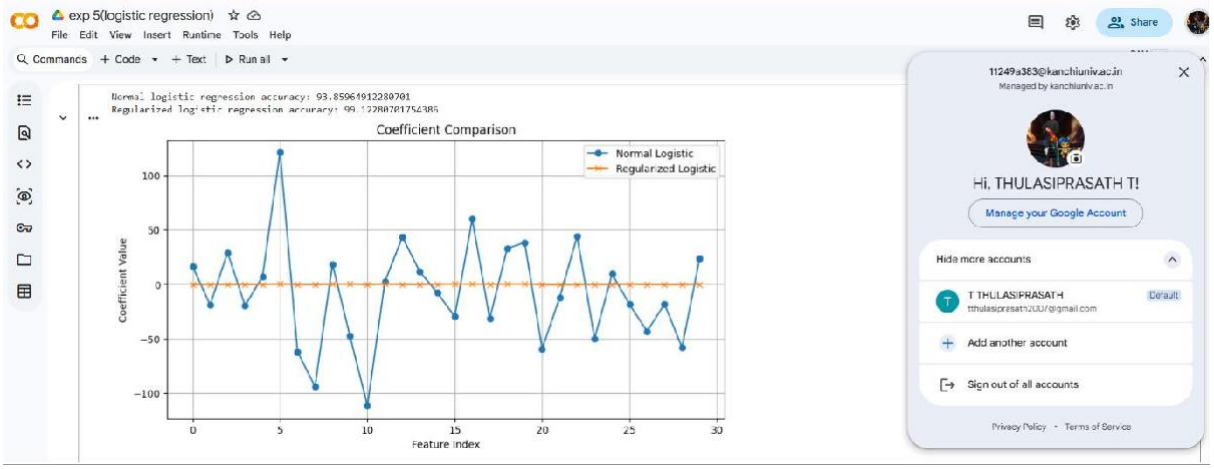
PROGRAM:

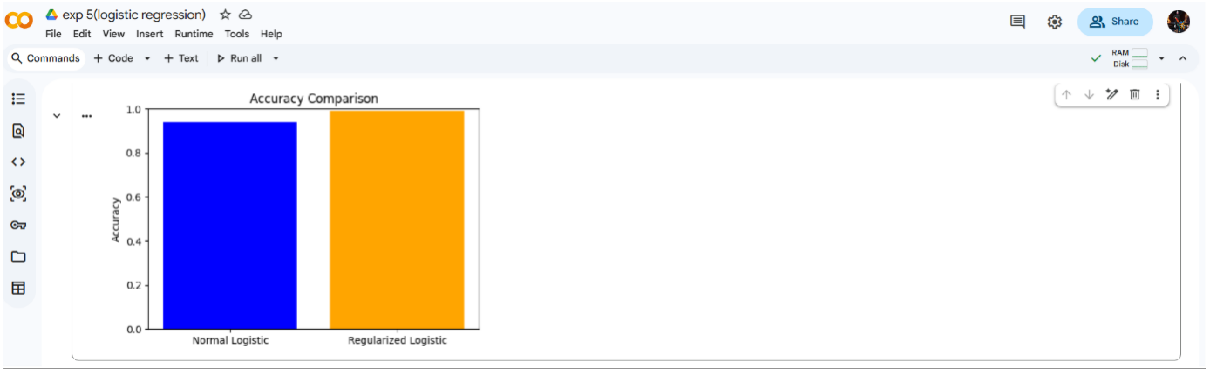


```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load data
data = load_breast_cancer()
X = data.data
y = data.target

# Split into train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Normal logistic regression (almost no regularization)
normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)
normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)
```

```
# Regularized logistic regression
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)

# Print accuracies
print("Normal logistic regression accuracy:", normal_acc * 100)
print("Regularized logistic regression accuracy:", reg_acc * 100)

# Plot coefficients
plt.figure(figsize=(10,5))
plt.plot(normal_model.coef_.flatten(), label='Normal Logistic', marker='o')
plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic', marker='x')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.grid(True)
plt.show()

# Plot accuracies
plt.figure(figsize=(5,4))
models = ['Normal Logistic', 'Regularized Logistic']
accuracies = [normal_acc, reg_acc]
plt.bar(models, accuracies, color=['blue', 'orange'])
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.ylim(0,1)
plt.show()
```

OUTPUT:





RESULT :

Successfully built a classification model with controlled overfitting and good accuracy.

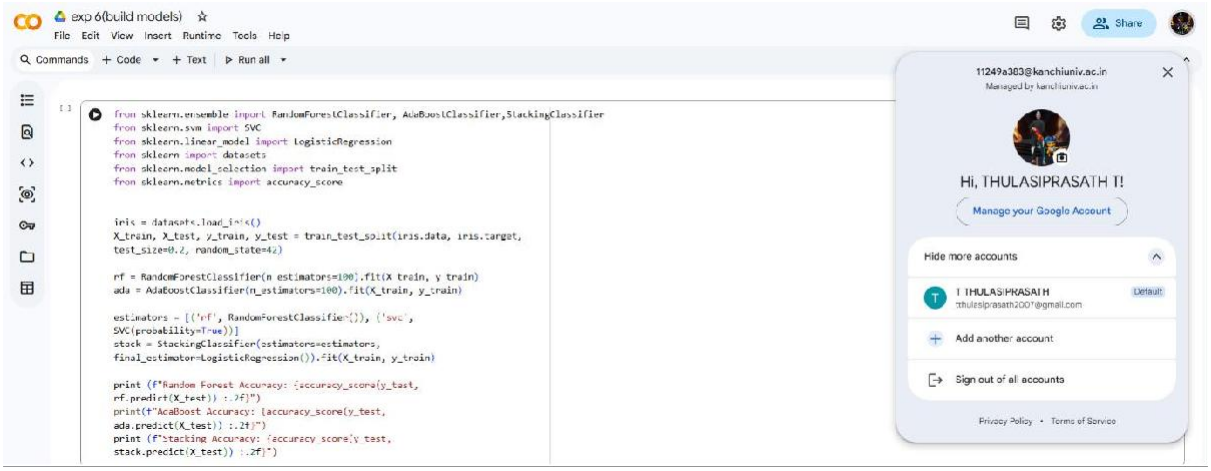
Exp.no: 06
Date : 09.03.2026

BUILD MODELS USING DIFFERENT ENSEMBLING TECHNIQUES.

AIM:

To build models using ensemble techniques like bagging and boosting.

PROGRAM:



```
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score


iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
test_size=0.2, random_state=42)

rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)

estimators = [('rf', RandomForestClassifier()), ('svc',
SVC(probability=True))]
stack = StackingClassifier(estimators=estimators,
final_estimator=LogisticRegression()).fit(X_train, y_train)

print (f"Random Forest Accuracy: {accuracy_score(y_test,
rf.predict(X_test)) :.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test,
ada.predict(X_test)) :.2f}")
print (f"Stacking Accuracy: {accuracy_score(y_test,
stack.predict(X_test)) :.2f}")
```

OUTPUT:

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

RESULT:

Successfully Improved model performance by combining multiple learning algorithms.

Exp.no: 07
Date : 09.03.2026

BUILD MODELS USING DECISION TREES.

AIM:

To implement a decision tree model for classification or regression.

Program:

OUTPUT:

RESULT:

Constructed a tree-based model for accurate prediction and decision-making.

Exp.no: 08
Date : 16.03.2026

SUPPORT VECTOR MACHINE (SVM)WITH DIFFERENT KERNELS.

AIM:

To implement models using different kernel functions in SVM.

PROGRAM:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Linear Kernel
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)

pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)

print("Linear Kernel Accuracy:", acc_linear)

# Polynomial Kernel
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)

pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
```

```
print("Polynomial Kernel Accuracy:", acc_poly)

# RBF Kernel
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)

pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)

print("RBF Kernel Accuracy:", acc_rbf)
```

OUTPUT:

```
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)

print("RBF Kernel Accuracy:", acc_rbf)

Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 1.0
```

RESULT:

Successfully applied kernel functions to improve classification performance.

Exp.no: 09
Date : 30.03.2026

IMPLEMENT K-NEAREST NEIGHBORS (KNN) CLASSIFICATION.

AIM:

To implement KNN algorithm for classification tasks.

PROGRAM:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
data = iris.frame

X = data.drop("target", axis=1)
y = data["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

OUTPUT:

```
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

RESULT:

Built a KNN model capable of classifying data based on nearest neighbors.

Exp.no: 10
Date : 30.03.2026

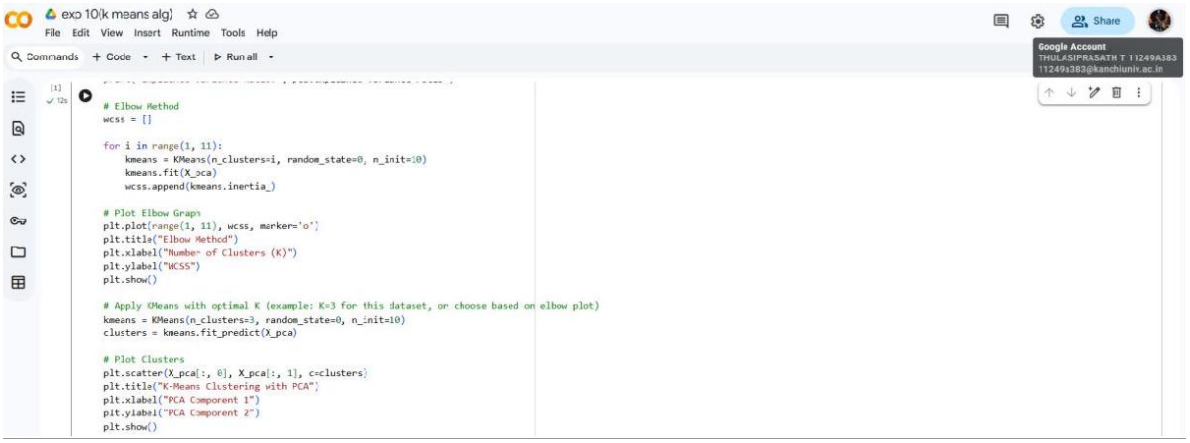
K-MEANS CLUSTERING WITH PCA AND ELBOW METHOD.

AIM:

To implement K-Means clustering and optimize clusters using PCA and Elbow Method.

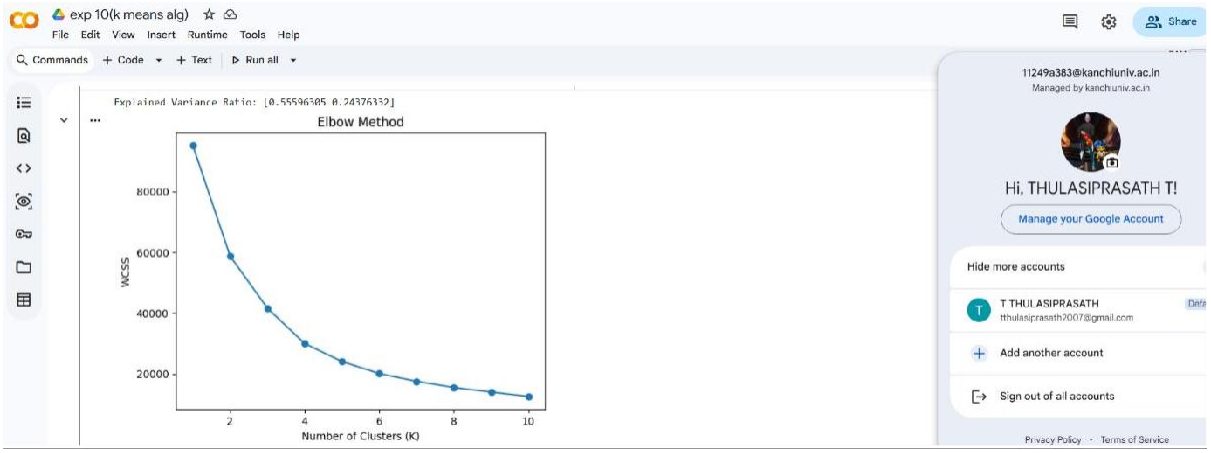
PROGRAM:

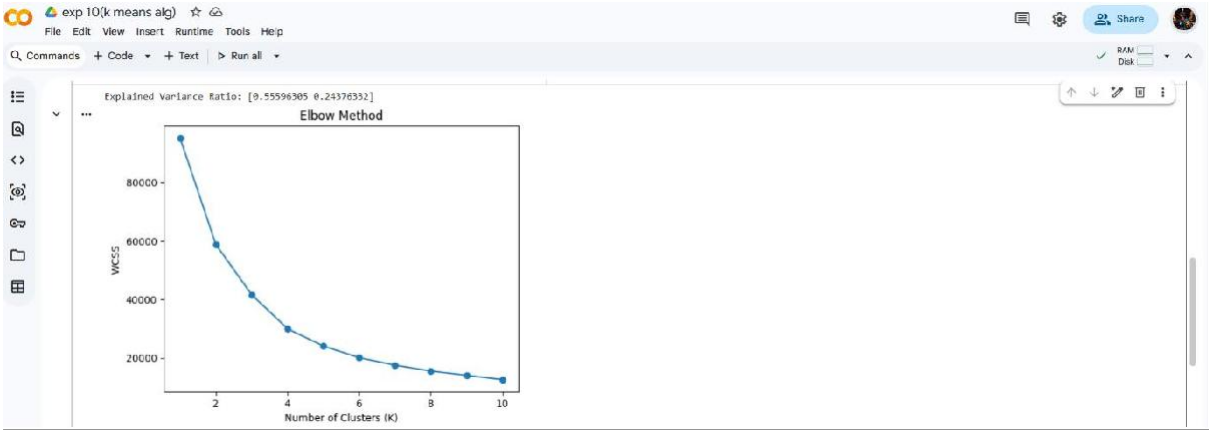
```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# Load dataset from sample_data
data = pd.read_csv("/content/sample_data/california_housing_train.csv")

# Select numerical features relevant to the new dataset
X = data[['housing_median_age', 'total_rooms', 'total_bedrooms', 'population', 'households', 'median_income', 'median_house_value']]

# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

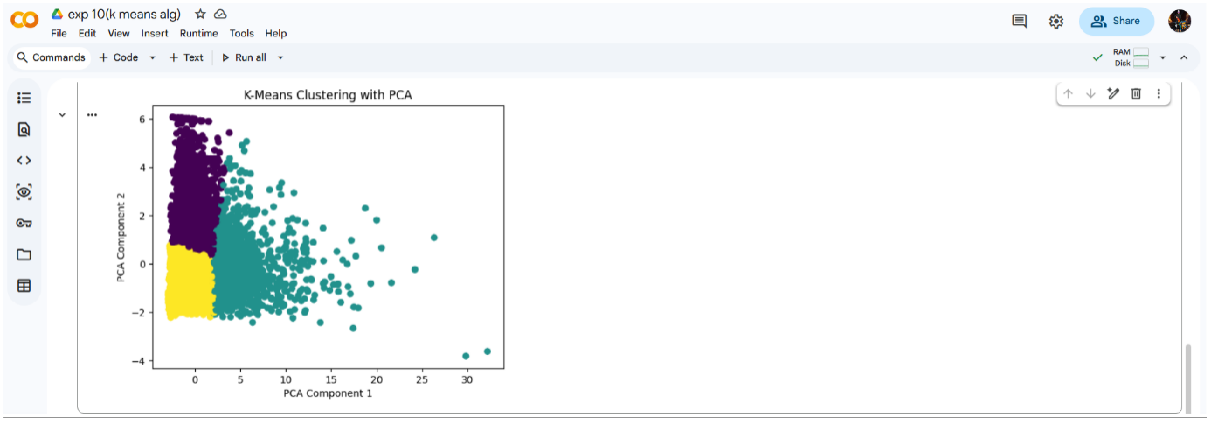


```
# Elbow Method
wcss = []

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=0, n_init=10)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)

# Plot Elbow Graph
plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()

# Apply KMeans with optimal K (example: K=3 for this dataset, or choose based on elbow plot)
kmeans = KMeans(n_clusters=3, random_state=0, n_init=10)
clusters = kmeans.fit_predict(X_pca)

# Plot Clusters
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters)
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

OUTPUT:







RESULT:

Successfully grouped data into clusters and reduced dimensionality for better visualization.